🐍 Python Class Note: Data Types (Primitive and Reference)

1️⃣ Introduction to Data Types

In Python, **data types** define the type of value a variable can hold and how the interpreter stores and manipulates it in memory.

Python is a **dynamically typed** language — you don't need to declare the type of variable explicitly.

```
x = 10       # int
y = "Hello"  # str
```

2️⃣ Categories of Data Types

Python data types are generally classified into two main categories:

1. **Primitive (Basic) Data Types**
2. **Reference (Non-Primitive / Complex) Data Types**

3️⃣ Primitive Data Types

Primitive data types store **simple values** that are not composed of other data.
They are **immutable** — their values cannot be changed once created.

A. Integer (`int`)

- Represents whole numbers (positive, negative, or zero).
- Example:

```
age = 25
temperature = -10
```

- Operations:

```
a = 10
b = 3
print(a + b)  # 13
print(a // b) # 3 (Floor Division)
```

B. Float (`float`)

- Represents decimal or fractional numbers.
- Example:

```
price = 12.99
weight = 70.5
```

- Operations:

```
print(3.5 + 2.1)  # 5.6
print(7 / 2)      # 3.5
```

C. Boolean (`bool`)

- Represents **True** or **False** values.
- Example:

```
is_logged_in = True
is_admin = False
```

- Used in conditions:

```
if is_logged_in:
    print("Welcome back!")
```

D. String (`str`)

- Represents a sequence of characters enclosed in quotes.
- Strings are **immutable**.
- Example:

```
name = "Joel"
greeting = 'Hello'
```

- String operations:

```
print(name.upper())      # "JOEL"
print(len(greeting))     # 5
print(name + " " + greeting)  # "Joel Hello"
```

E. None Type (`NoneType`)

- Represents **absence of value**.
- Example:

```
result = None
print(result)  # Output: None
```

4️⃣ Reference (Non-Primitive) Data Types

Reference data types store **complex structures** and **references** (memory addresses) of the actual data.
They are **mutable** in most cases.

A. List

- Ordered, mutable collection of items.
- Example:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits)
```

- Access:

```
print(fruits[0])  # apple
```

B. Tuple

- Ordered, immutable collection of items.
- Example:

```
coordinates = (10, 20)
print(coordinates[1])  # 20
```

C. Set

- Unordered collection of unique items.
- Example:

```
colors = {"red", "blue", "green"}
colors.add("yellow")
print(colors)
```

- Removes duplicates automatically.

D. Dictionary (`dict`)

- Stores key-value pairs.
- Example:

```
student = {
    "name": "Joel",
    "age": 25,
    "course": "Engineering"
}
print(student["name"])  # Joel
```

- Add or modify:

```
student["grade"] = "A"
```

E. Complex Numbers (`complex`)

- Used for mathematical operations involving imaginary numbers.
- Example:

```
z = 3 + 4j
print(z.real)  # 3.0
print(z.imag)  # 4.0
```

5️⃣ Difference Between Primitive and Reference Data Types

Feature	Primitive	Reference
Stored Value	Holds actual value	Holds reference (address)
Mutability	Immutable	Mostly mutable
Examples	int, float, bool, str	list, dict, set, tuple
Memory Storage	Stored in stack	Stored in heap
Assignment	Copy by value	Copy by reference

Example:

```
# Primitive
x = 5
y = x
y = 10
print(x)  # 5 (unchanged)

# Reference
list1 = [1, 2, 3]
list2 = list1
list2.append(4)
print(list1)  # [1, 2, 3, 4] (changed)
```

6️⃣ Type Conversion (Casting)

You can convert between data types using built-in functions:

```
# int to float
x = 10
y = float(x)
print(y)  # 10.0

# str to int
num = int("25")
print(num + 5)  # 30

# list to set
lst = [1, 2, 2, 3]
st = set(lst)
print(st)  # {1, 2, 3}
```

7 Type Checking

Use the `type()` function to check the data type of a variable.

```
name = "Joel"
print(type(name))  # <class 'str'>
```

8 Summary

Category	Data Type	Mutable	Example
Primitive	int	❌	`10`
Primitive	float	❌	`10.5`
Primitive	bool	❌	`True`
Primitive	str	❌	`"Hello"`
Reference	list	✅	`[1, 2, 3]`
Reference	tuple	❌	`(1, 2, 3)`
Reference	set	✅	`{1, 2, 3}`
Reference	dict	✅	`{"name": "Joel"}`

9 Class Exercise

Q1. Create a dictionary to store your name, age, and favorite programming language.

Q2. Convert the dictionary keys to a list.

Q3. Demonstrate the difference between a mutable and immutable type using list and tuple.